МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Лабораторная работа 5. Обработка последовательностей

Студент
гр. БИН-26-1 _____ Н.С. Коростелёв

Преподаватель _____ М.В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Обратный отсчет

В задании необходимо вывести все целые числа от введённого числа n до 0 включительно в порядке убывания.

Изображение схемы решения первого задания 1.

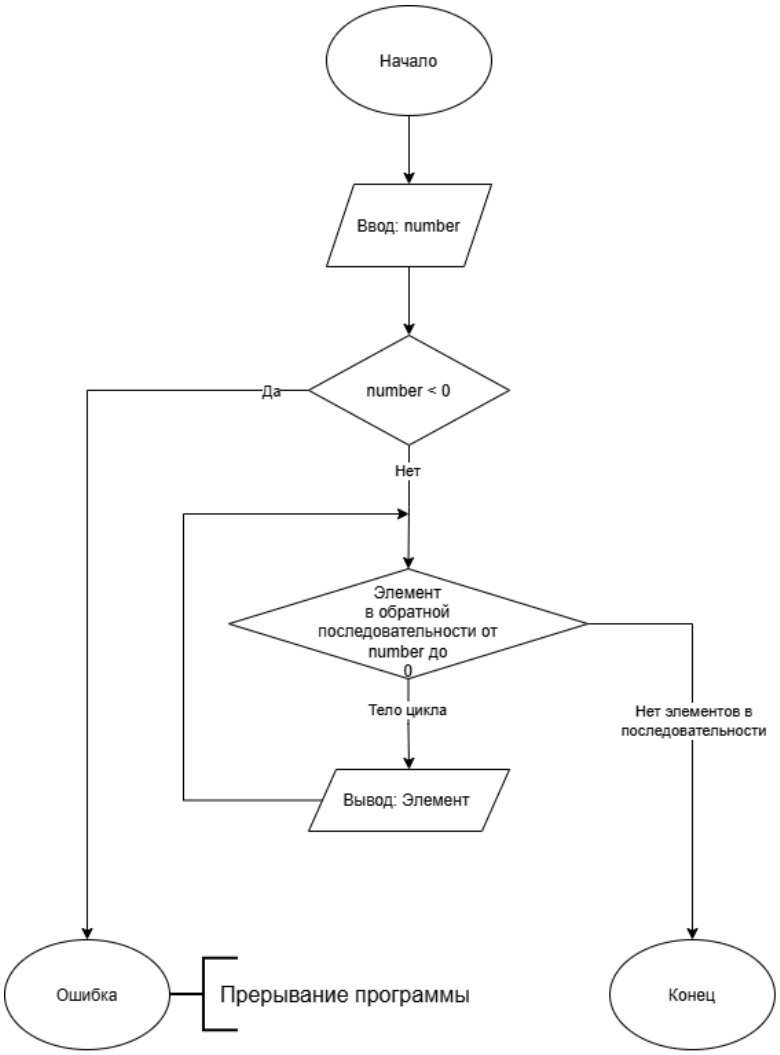


Рисунок 1 – Схема решения первого задания

Программа с помощью цикла for последовательно итерируется по последовательности range, выводя значения на новых строках.

Изображение кода для первого задания 2.

```python
def main():
    number = int(input())

    if not(number > 0):
        raise ValueError("Not valid value")

    for i in range(number, -1, -1):
        print(i)

if __name__ == "__main__":
    main()
```

Рисунок 2 – Код первого задания

Отсчёт выполняется корректно.

1.2 Задание 2. Сумма и произведение

В задании необходимо вычислить сумму и произведение всех целых чисел от a до b включительно.

Изображение схемы решения второго задания 3.

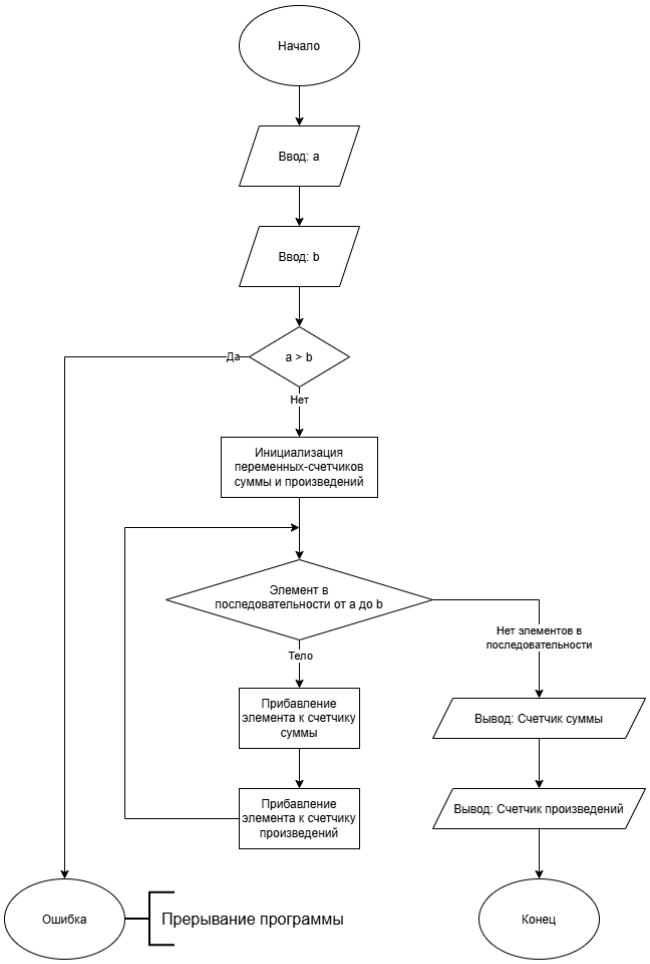


Рисунок 3 – Схема решения второго задания

Программа считывает границы диапазона и с помощью цикла for накапливает сумму и произведение чисел. Результат выводится в требуемом формате.

Изображение кода для второго задания 4.

```
def main():
    a = int(input())
    b = int(input())

    if not(a <= b):
        raise ValueError("Not valid number")

    summary = 0
    multiplication = 1

    for i in range(a, b + 1):
        summary += i
        multiplication *= i

    print(f"Сумма: {summary}")
    print(f"Произведение: {multiplication}")

if __name__ == "__main__":
    main()
```

Рисунок 4 – Код второго задания

Сумма и произведение вычисляются корректно.

1.3 Задание 3. Подсчет измерений

В задании необходимо подсчитать количество значений температуры ниже нуля, равных нулю и выше нуля.

Изображение схемы решения третьего задания 5.

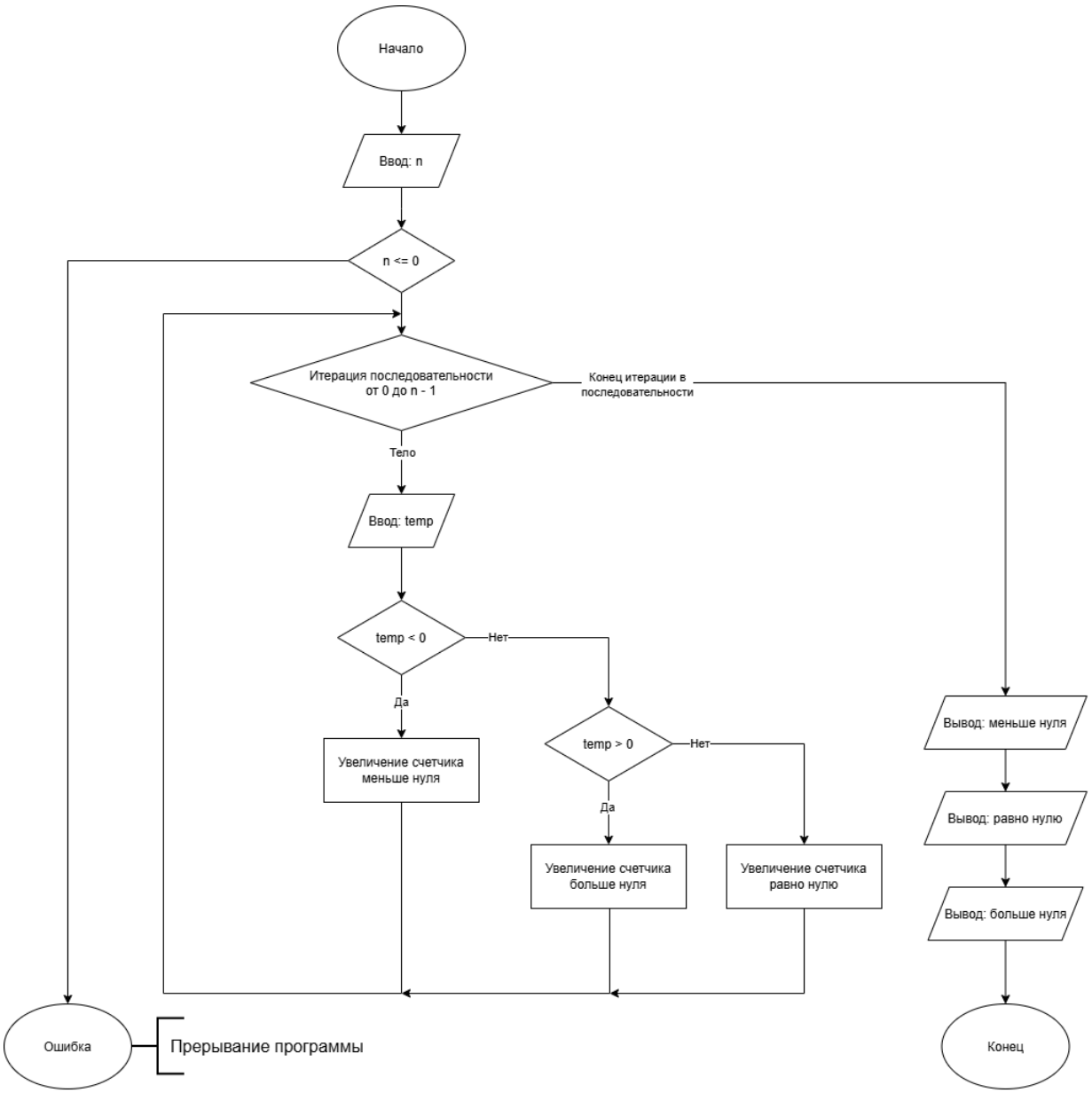


Рисунок 5 – Схема решения третьего задания

Программа считывает количество измерений и затем обрабатывает каждое значение, увеличивая соответствующий счётчик. Если $n = 0$, все счётчики остаются равны нулю.

Изображение кода для третьего задания 6.

```python
def main():
    n = int(input())

    if not(n > 0):
        raise ValueError("Not valid value")

    below_zero = 0
    equal_zero = 0
    above_zero = 0

    for _ in range(n):
        temp = float(input())
        if temp < 0:
            below_zero += 1
        elif temp > 0:
            above_zero += 1
        else:
            equal_zero += 1

    print(f"Ниже нуля: {below_zero}")
    print(f"Равны нулю: {equal_zero}")
    print(f"Выше нуля: {above_zero}")

if __name__ == "__main__":
    main()
```

Рисунок 6 – Код третьего задания

Подсчёт измерений выполняется корректно.

1.4 Задание 4. Таблица умножения

В задании необходимо вывести таблицу умножения заданного числа от 1 до 10.

Изображение схемы решения четвёртого задания 7.

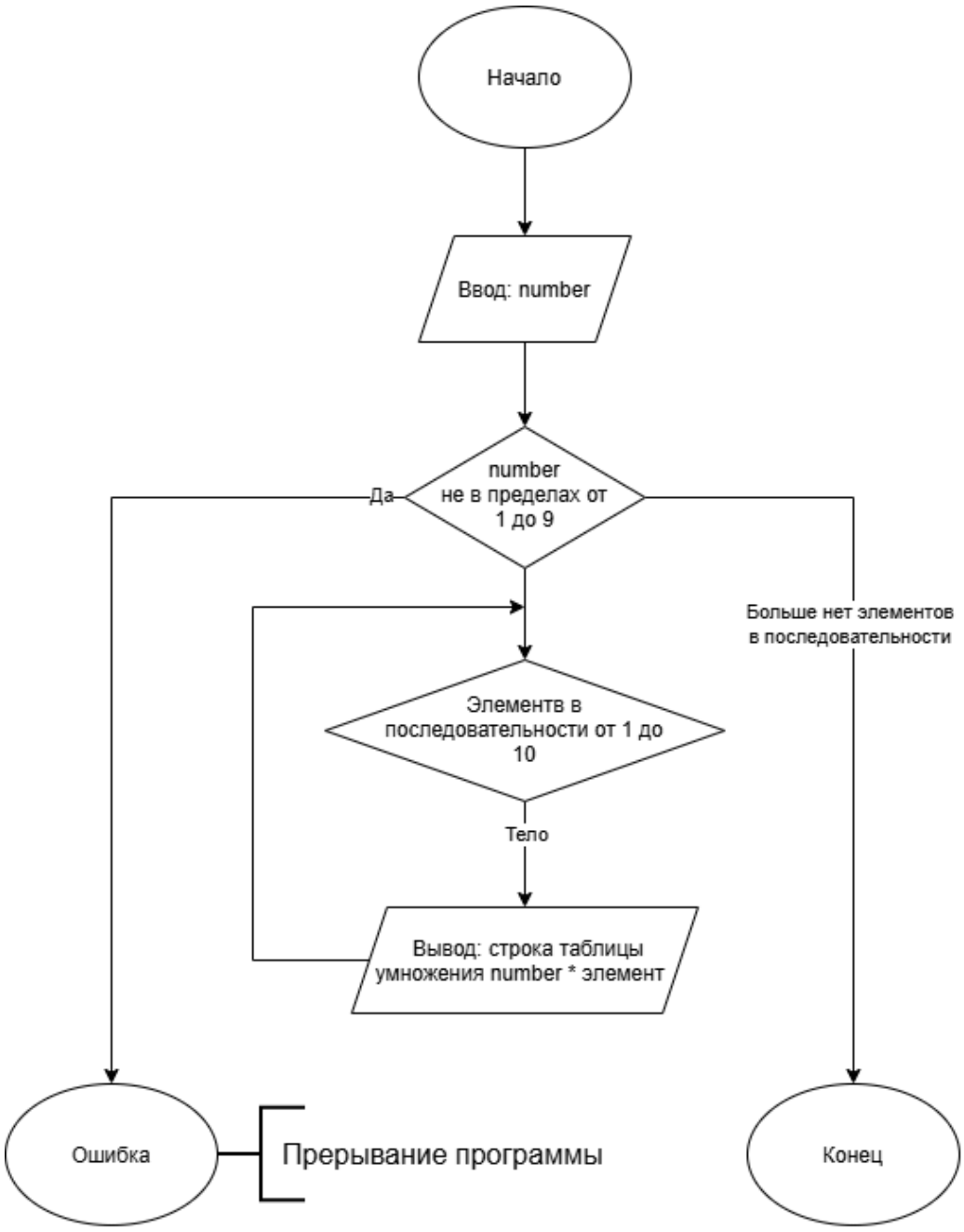


Рисунок 7 – Схема решения четвертого задания

Программа с помощью цикла for формирует строки таблицы умножения и выводит произведения в порядке возрастания множителя.

Изображение кода для четвертого задания 8.

```python
def main():
    number = int(input())

    if not(1 <= number <= 9):
        raise ValueError("Not valid value")

    multiplicand = number
    for multiplier in range(1, 10 + 1):
        print(f"{multiplicand} * {multiplier} = "
              f"{multiplicand * multiplier}")

if __name__ == "__main__":
    main()
```

Рисунок 8 – Код четвертого задания

Таблица умножения построена верно.

1.5 Задание 5. Учет покупок

В задании необходимо считать количество покупок, их общую стоимость и среднюю стоимость одной покупки. Ввод завершается командой stop.

Изображение схемы решения пятого задания 9.

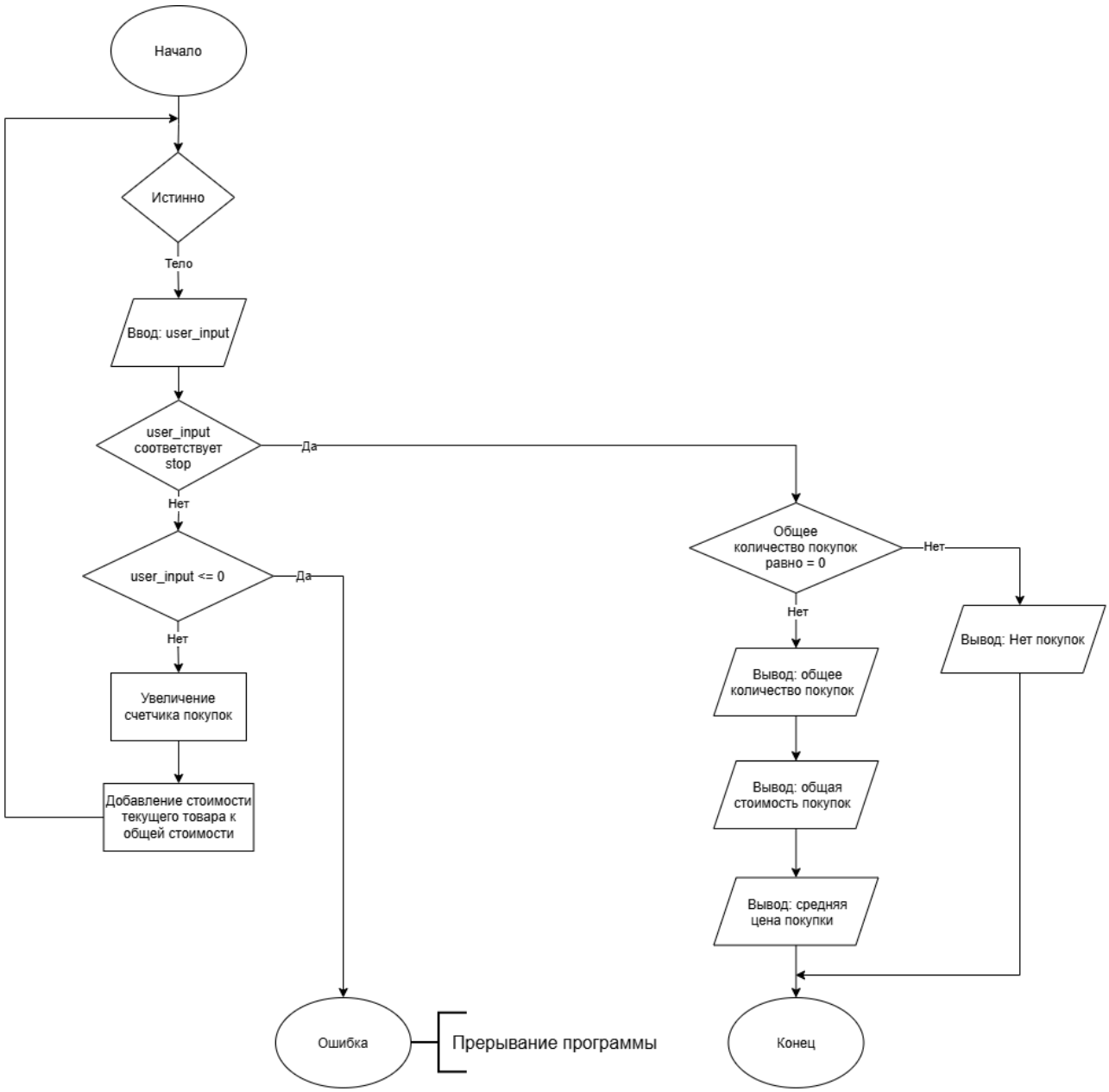


Рисунок 9 – Схема решения пятого задания

Программа считывает значения покупок до ввода команды stop, накапливает их количество и сумму, а затем вычисляет среднюю стоимость. Если покупок нет, выводится сообщение Нет покупок.

Изображение кода для пятого задания 10.

```python
def main():
    purchases_count = 0
    total_cost = 0

    while True:
        user_input = input()

        if user_input == "stop":
            break

        purchase_cost = int(user_input)

        if not(purchase_cost > 0):
            raise ValueError("Not valid value")

        purchases_count += 1
        total_cost += purchase_cost

    if purchases_count == 0:
        print("Нет покупок")

    else:
        average = total_cost / purchases_count
        print(f"Покупок: {purchases_count}")
        print(f"Общая стоимость: {total_cost} руб")
        print(f"Средняя стоимость: {average:.2f} руб")

if __name__ == "__main__":
    main()
```

Рисунок 10 – Код пятого задания

Учёт покупок выполняется корректно.

1.6 Задание 6. Треугольник Паскаля

В задании необходимо вывести первые n строк треугольника Паскаля и подсчитать количество нечётных элементов во всех выведенных строках.

Изображение схемы решения шестого задания 11.

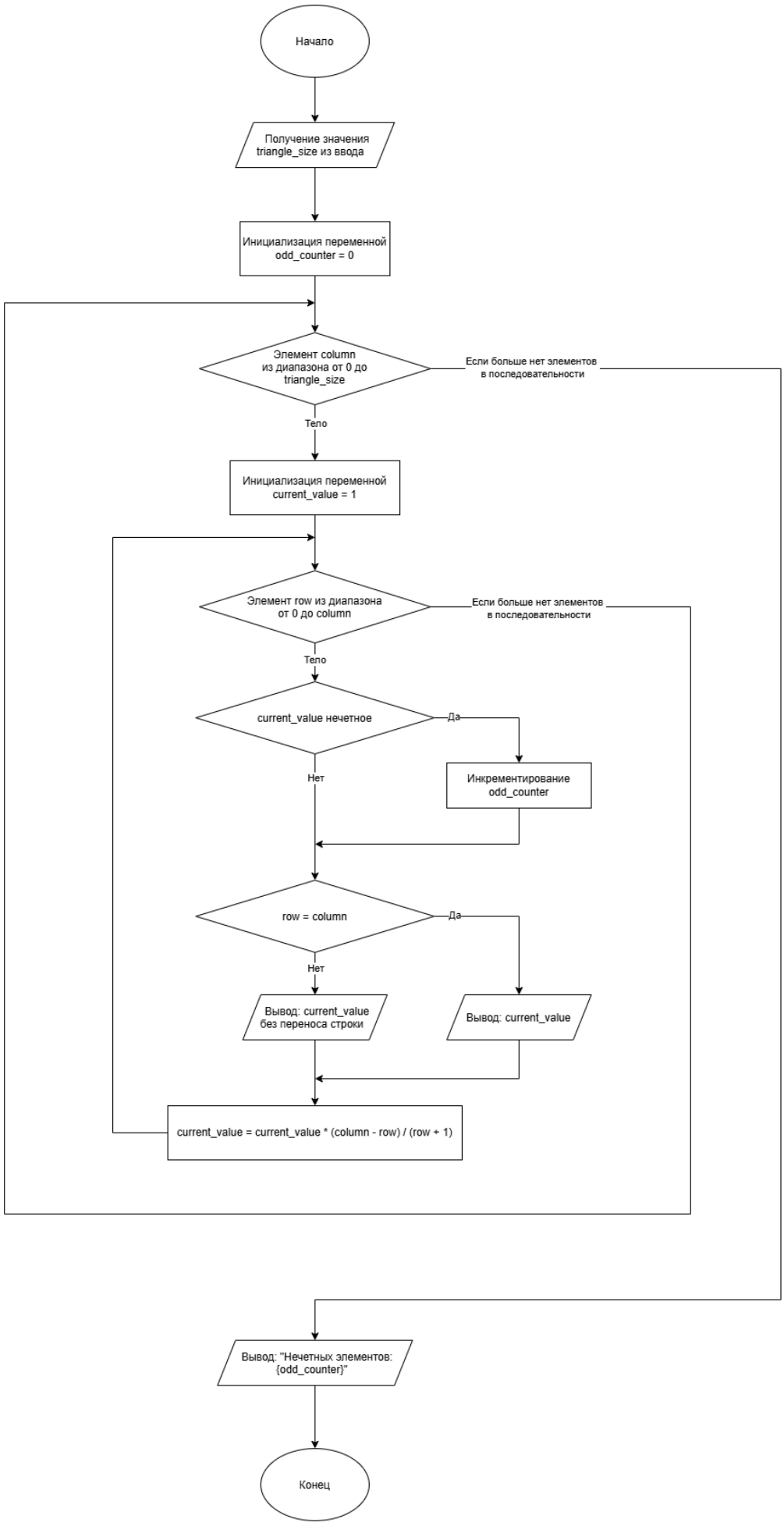


Рисунок 11 – Схема решения шестого задания

Программа формирует строки треугольника Паскаля с помощью вложенных циклов и целочисленных вычислений. После вывода всех строк выводится количество нечётных элементов.

Изображение кода для шестого задания 12.

```
def main():
    triangle_size = int(input())
    odd_counter = 0

    for column in range(triangle_size):

        current_value = 1

        for row in range(column + 1):

            if current_value % 2 != 0:
                odd_counter += 1

            if row == column:
                print(current_value)

            else:
                print(current_value, end=" ")

            current_value = current_value * (column - row) // (row + 1)

    print(f"Нечетных элементов: {odd_counter}")

if __name__ == "__main__":
    main()
```

Рисунок 12 – Код шестого задания

Треугольник Паскаля построен корректно, а количество нечётных элементов определено верно.